



[AE2] DESARROLLO Y PROFUNDIZACIÓN DE ARQUITECTURA Y COMUNICACIÓN ENTRE APLICACIONES

PARADIGMAS Y LENGUAJES DE PROGRAMACIÓN III

Breve Introducción

Actividad formativa individual orientada a profundizar la arquitectura distribuida, contratos, asincronía, persistencia y consistencia. Puede tomar como base un sistema provisto o una versión congelada de AE1, pero la evidencia y la implementación evaluada deben ser individuales.

Profundizar el diseño arquitectónico y la comunicación entre aplicaciones mediante servicios, APIs y mecanismos asíncronos, justificando decisiones de integración, datos, concurrencia, interoperabilidad y desacoplamiento en una solución distribuida.

Codificación de la Asignatura: ISI17PL324

Codificación de la Actividad: ISI17PL324V26AE2V26

Nombres, apellidos y DNI de los docentes integrantes de la cátedra:

{a completar por la catedra}

Firmas:

{a completar por la catedra}



CONSIGNA DE ACTIVIDAD DE EVALUACIÓN

CODIFICACIÓN DE LA ACTIVIDAD:	ISI17PL324V26AE2V26
SEDE:	{a completar por la cátedra}
FACULTAD:	FAITA
CARRERA: INGENIERÍA EN SISTEMA DE INFORMACIÓN	MODALIDAD ☒ Presencial ☐ Digital
NOMBRE DE LA ASIGNATURA: PARADIGMAS Y LENGUAJES DE PROGRAMACIÓN III	MODALIDAD ☒ Presencial ☐ Presencial Digital ☐ Presencial Mixta ☐ Digital
NOMBRE Y APELLIDO DEL DOCENTE:	{a completar por la cátedra}
NOMBRE Y APELLIDO DEL/LOS ESTUDIANTES:	{a completar por la cátedra}
DNI DEL/LOS ESTUDIANTES:	{a completar por la cátedra}
FECHA Y HORARIO DE REALIZACIÓN Y/O DE ENTREGA: {a completar por la cátedra}	

TIPO DE EVALUACIÓN:	☐ Actividad de Evaluación 1 ☒ Actividad de Evaluación 2 ☐ Actividad de Evaluación 3 ☐ Actividad de Evaluación 4	
MODALIDAD DE INSTANCIA EVALUATIVA:	☒ Individual ☐ Pares ☐ Tríos	☐ Sincrónica ☒ Asincrónica
CONTENIDOS A EVALUAR: Enunciar los contenidos involucrados en la evaluación.		



- Unidad/Eje 3: Arquitectura y comunicación entre Aplicaciones: estilos, cliente/servidor, servicios, APIs RESTful, comunicación síncrona/asíncrona, eventos/callbacks/promesas, SOA, integración e interoperabilidad.
- Contenidos previos de U1/U2 necesarios para resolver concurrencia, distribución y construcción de aplicaciones web.

CRITERIOS DE EVALUACIÓN:

- Evolución trazable desde AE1
- Cumplimiento de los RF individuales comprometidos
- Calidad de la evolución arquitectónica
- Contratos REST y comunicación asíncrona
- Uso pertinente de Redis y RabbitMQ
- Concurrencia, consistencia e idempotencia
- Propiedad de datos
- Testing y reproducibilidad
- Trazabilidad individual mediante branch/commits/Portafolio
- Fundamentación de las decisiones técnicas.

Criterios transversales

Se valorará claridad conceptual, capacidad de análisis, selección de información pertinente, aplicación de los conceptos al caso, fundamentación de decisiones, producción escrita clara y uso adecuado de vocabulario técnico. La ortografía y redacción se considerarán conforme a los criterios institucionales y de la asignatura.

INSTRUMENTO DE EVALUACIÓN:

Se utilizará una rúbrica analítica de carácter formativo para valorar el producto presentado y las evidencias del proceso de aprendizaje.

La rúbrica no asigna una calificación numérica disciplinar. Su finalidad es determinar el grado de desarrollo de las competencias previstas para AE2 y proporcionar feedback que permita revisar y mejorar la solución.

Se utilizarán los siguientes niveles:

- **Logrado:** la evidencia permite verificar de manera suficiente, coherente y fundamentada el desempeño esperado para AE.
- **En desarrollo:** el desempeño esperado se encuentra mayormente presente, aunque requiere ajustes o profundización o mejor evidencia.
- **Requiere revisión:** existen evidencias parciales, inconsistencias conceptuales o



técnicas, o aspectos sustanciales que deben reelaborarse.

- **Sin evidencia:** no se dispone de evidencia suficiente para verificar el desempeño.

Criterio	Logrado	En desarrollo	Requiere revisión	Sin evidencia
1. Trazabilidad de la evolución AE1 → AE2	Identifica claramente la versión base de AE1, los módulos heredados, los RF seleccionados y el alcance individual. Puede diferenciarse con precisión qué existía previamente y qué fue desarrollado en AE2.	La evolución puede reconstruirse, aunque existen algunos cambios o requerimientos cuya procedencia no queda completamente clara.	Resulta difícil distinguir la solución heredada de la producción individual o el alcance comprometido no está suficientemente definido.	No existe una referencia verificable a la versión de AE1 ni evidencia de la evolución individual.
2. Implementación de los requerimientos funcionales comprometidos	Los RF seleccionados están implementados, integrados con la solución heredada y producen resultados verificables acordes al escenario.	La mayoría funciona correctamente, aunque existen restricciones o defectos menores.	La implementación es parcial, presenta defectos relevantes o no mantiene coherencia con el escenario.	Los RF comprometidos no están implementados o no pueden verificarse.
3. Evolución arquitectónica y límites de responsabilidad	Analiza la arquitectura heredada y realiza mejoras justificadas en cohesión,	La arquitectura presenta una evolución válida, aunque algunas responsabilidades	Los cambios son principalmente funcionales y no	No existe evidencia de análisis o evolución arquitect



	desacoplamiento, responsabilidades y límites entre servicios o módulos.	des o dependencias requieren mayor revisión.	evidencian comprensión de los límites arquitectónicos o generan nuevo acoplamiento o innecesario.	ónica.
4. APIs, contratos y comunicación síncrona	Los contratos son claros y consistentes; las responsabilidades están delimitadas y las APIs se encuentran correctamente documentadas cuando corresponde.	Los contratos permiten operar el sistema, aunque existen inconsistencias menores o documentación incompleta.	Las comunicaciones poseen contratos ambiguos, acoplamiento excesivo o manejo deficiente de errores.	No existen contratos verificables para las comunicaciones implementadas.
5. Comunicación asíncrona y mensajería	Utiliza RabbitMQ o mecanismo aprobado de forma coherente con el problema; identifica productores, consumidores, eventos y comportamiento ante reintentos, demostrando desacoplamiento real.	La mensajería funciona pero su diseño, manejo de errores o justificación requiere mejoras.	Se utiliza mensajería sin una necesidad arquitectónica clara, de forma incompleta o con fuerte acoplamiento.	No existe la comunicación asíncrona requerida para el alcance seleccionado.
6. Redis, caché y estado efímero	Redis o mecanismo aprobado se utiliza en un	El mecanismo funciona pero faltan evidencias o	Redis se utiliza como simple almacenami	No se implementa el requisito



	caso pertinente; demuestra lectura/escritura y, cuando corresponde, TTL, expiración e invalidación.	existen decisiones de expiración/invalididad poco justificadas.	ento sin relación clara con el problema o su gestión de estado es incorrecta.	cuando forma parte del alcance obligatorio de AE2.
7. Concurrencia, consistencia e idempotencia	Identifica un caso realista de carrera o repetición, demuestra el problema y verifica que la solución evita duplicaciones, estados inválidos o efectos repetidos.	Existe tratamiento correcto en el caso principal, pero faltan pruebas suficientes o análisis de situaciones límite.	El problema está identificado pero puede producir todavía inconsistencias o duplicaciones.	No existe evidencia de tratamiento del problema requerido.
8. Persistencia, propiedad de datos e interoperabilidad	Cada servicio posee responsabilidad clara sobre sus datos; evita accesos indebidos entre servicios y justifica la estrategia de persistencia e integración.	La separación es mayormente correcta, aunque existen dependencias o decisiones que requieren mejor fundamentación.	Existen lecturas directas de datos ajenos, fuerte acoplamiento o entre esquemas o decisiones sin justificar.	No se documenta ni puede verificarse la estrategia de datos.
9. Integraciones, QR, PDF u otros artefactos requeridos por el escenario	Los elementos requeridos están correctamente integrados o simulados, poseen manejo básico de errores y cumplen la	Funcionan en el flujo principal, aunque presentan limitaciones menores o documentación	Su implementación es parcial, no está integrada al recorrido o presenta problemas	No existe evidencia del elemento requerido para el alcance seleccion



	finalidad definida en el escenario.	n insuficiente.	relevantes.	ado.
10. Testing y reproducibilidad	Existen pruebas significativas de los comportamientos incorporados y de casos críticos. La solución puede ejecutarse de forma reproducible siguiendo la documentación proporcionada.	Las pruebas cubren el flujo principal, pero faltan casos relevantes o existen pasos manuales de ejecución no suficientemente documentados.	Las pruebas son superficiales o la solución depende fuertemente del entorno personal del estudiante.	No existen pruebas verificables o no puede ejecutarse la solución.
11. Trazabilidad individual mediante Git y Portafolio	El branch individual, commits, tareas/issues y Portafolio permiten reconstruir claramente el trabajo realizado por el estudiante y su evolución temporal.	La autoría individual es identificable aunque existen algunos cambios o decisiones con trazabilidad incompleta.	La contribución individual resulta difícil de diferenciar del código heredado o las evidencias son escasas.	No existe evidencia suficiente para establecer la autoría individual.
12. Fundamentación y reflexión profesional	Compara alternativas, fundamenta las decisiones principales utilizando conceptos de la asignatura y reconoce ventajas,	Las decisiones son razonables, aunque la comparación o fundamentación necesita mayor profundidad.	Predomina una explicación descriptiva de tecnologías sin relacionarlas con el problema o	No se presenta fundamentación de las decisiones realizadas.



	limitaciones y compromisos de la solución adoptada.		los conceptos trabajados.	
Relación con Saber, Saber Hacer y Saber Ser La rúbrica también permite verificar las tres dimensiones • Saber: criterios 3, 4, 5, 6, 7, 8 y 12, mediante comprensión de estilos de arquitectura, REST, asincronía, integración, persistencia, concurrencia e interoperabilidad. • Saber Hacer: criterios 2, 4, 5, 6, 7, 9 y 10, mediante implementación de RF, APIs, mensajería, Redis, idempotencia, integraciones, testing y ejecución reproducible. • Saber Ser / Experiencia Profesional: criterios 1, 11 y 12, junto con las evidencias de todo el proceso: autonomía, responsabilidad sobre una base compartida, trazabilidad, fundamentación de decisiones y comunicación técnica. La Guía exige que la secuencia produzca evidencia de estas tres dimensiones.				
CONSIGNA DE ACTIVIDAD DE EVALUACIÓN: La Actividad de Evaluación 2 (AE2) constituye una instancia individual de evolución, desarrollo y profundización del trabajo iniciado en la AE1. Cada estudiante deberá continuar trabajando sobre el mismo escenario y la misma solución de software en la que participó durante AE1 junto con su pareja o trío, tomando como punto de partida la versión oficialmente entregada y aceptada por la cátedra El propósito de AE2 es que cada estudiante pueda demostrar individualmente su capacidad para comprender una solución previamente desarrollada en equipo, analizar su arquitectura, seleccionar una parte significativa del sistema, evolucionarla funcional y técnicamente y fundamentar las decisiones adoptadas, incorporando los conocimientos correspondientes a arquitectura y comunicación entre aplicaciones. La actividad no consiste en reconstruir desde cero el trabajo de AE1 ni en desarrollar una solución independiente. Se busca realizar una evolución controlada de la misma				



aplicación, conservando la trazabilidad entre ambas actividades:

- AE1: Versión grupal 1.0
- AE2: Evolución individual 2.0
- AE4: Integración y evolución final

Punto de partida de la actividad

La cátedra establecerá una versión base de AE1, identificada mediante un tag, release, commit o mecanismo equivalente en el repositorio Git.

Cada estudiante deberá partir de esa misma versión y desarrollar AE2 en un **branch individual**, separado del branch principal utilizado por el equipo.

A modo de convención, podrá utilizarse una nomenclatura similar a:

- *ae2/<usuario-o-apellido>*

Por ejemplo:

- *ae2/juan-perez*
- *ae2/maria-fernandez*

Durante el desarrollo de AE2:

- el estudiante no deberá realizar el trabajo directamente sobre main, master o el branch principal equivalente
- los commits correspondientes a AE2 deberán ser identificables como producción individual
- las tareas, issues, commits, pruebas y documentación deberán permitir reconstruir el trabajo realizado
- la integración posterior del branch al producto común se realizará cuando la cátedra lo determine

Esta organización del repositorio tiene como finalidad preservar el producto desarrollado en AE1 y disponer de evidencia objetiva de la evolución individual realizada en AE2.

Definición del alcance individual de AE2

Antes de iniciar la implementación, cada estudiante deberá identificar y documentar el **alcance individual comprometido para AE2**.

Para ello deberá seleccionar, de acuerdo con los lineamientos del escenario y con aprobación de la cátedra, un conjunto de **requerimientos funcionales**



pertenecientes a uno o más módulos que su equipo haya implementado o iniciado durante AE1.

La selección deberá permitir realizar una evolución significativa del sistema y demostrar las competencias específicas previstas para AE2.

El estudiante deberá documentar como mínimo:

- escenario sobre el que trabaja
- versión de AE1 utilizada como punto de partida
- módulo o módulos seleccionados
- requerimientos funcionales que evolucionará o incorporará
- requerimientos no funcionales relacionados
- dependencias con otros componentes;
- estado inicial heredado de AE1
- alcance que se compromete a implementar individualmente en AE2
- funcionalidades que permanecerán fuera de esta entrega

El alcance seleccionado deberá ser **acotado, verificable y técnicamente significativo**.

No se valorará la cantidad de funcionalidades incorporadas sino la capacidad de analizarlas, diseñarlas, implementarlas, probarlas y fundamentarlas adecuadamente.

Desarrollo de la evolución

A partir del alcance individual definido, el estudiante deberá analizar la solución heredada de AE1 y realizar las modificaciones necesarias para evolucionar su arquitectura y comportamiento.

La implementación deberá permitir demostrar, según corresponda al escenario seleccionado:

- **Arquitectura y responsabilidades.** Revisar la separación de responsabilidades entre aplicaciones, módulos y servicios, identificando situaciones de acoplamiento innecesario y proponiendo mejoras que favorezcan cohesión, mantenibilidad e independencia de los componentes.
- **Comunicación entre aplicaciones.** Diseñar, implementar y documentar los mecanismos mediante los cuales los componentes intercambian información. El estudiante deberá analizar cuándo resulta apropiado utilizar comunicación síncrona y cuándo una comunicación asíncrona aporta ventajas al problema.
- **APIs y contratos.** Las comunicaciones síncronas deberán poseer contratos explícitos y coherentes. Cuando corresponda, las APIs deberán documentarse mediante OpenAPI o mecanismo equivalente aprobado por la cátedra.
- **Comunicación asíncrona.** Incorporar los mecanismos de mensajería definidos para AE2, identificando claramente productores, consumidores, mensajes o eventos, y justificando por qué determinados procesos pueden desacoplarse del



flujo principal.

- **Persistencia y propiedad de datos.** Analizar la estrategia de almacenamiento utilizada en AE1 y evolucionarla cuando corresponda. Cada servicio deberá declarar qué información administra y cómo accede a información perteneciente a otros servicios. No deberán utilizarse accesos directos a tablas pertenecientes a otros módulos cuando el diseño de AE2 establezca separación de responsabilidades.
- **Estado temporal y caché.** Incorporar Redis u otra tecnología expresamente aprobada por la cátedra en aquellos casos definidos por el escenario para administrar caché, datos temporales, expiraciones, turnos, disponibilidad, ofertas u otro estado efímero. El estudiante deberá demostrar y explicar su utilización, expiración e invalidación cuando corresponda.
- **Mensajería.** Incorporar RabbitMQ o mecanismo equivalente aprobado por la cátedra en los flujos definidos para AE2, demostrando la comunicación entre productor y consumidor y el comportamiento del sistema ante reintentos o procesamiento repetido.
- **Concurrencia, consistencia e idempotencia.** Identificar y resolver al menos una situación relevante en la que solicitudes concurrentes o repetidas puedan generar resultados incorrectos. El estudiante deberá demostrar el problema, explicar su posible consecuencia y justificar la solución implementada.

Ejemplos posibles son:

- dos usuarios intentando reservar el mismo recurso
- dos conductores intentando aceptar simultáneamente un mismo viaje
- aceptación y cancelación concurrentes
- procesamiento repetido de un mensaje
- confirmación duplicada de una operación
- generación duplicada de un pago o comprobante

Los ejemplos son orientativos; podrá utilizarse cualquier caso equivalente correspondiente al escenario asignado.

- **Integraciones y artefactos.** Cuando el alcance del escenario lo requiera, deberán incorporarse mecanismos tales como generación y validación de QR, generación de documentos PDF, consumo de servicios externos, mapas, pagos, notificaciones u otras integraciones. Podrán utilizarse servicios reales, sandbox, mocks o simulaciones cuando sean aprobados por la cátedra.
- **Tratamiento de errores.** Las comunicaciones e integraciones deberán contemplar condiciones de error previsibles, evitando que un fallo externo produzca comportamientos inconsistentes o errores no controlados.
- **Testing.** La evolución deberá estar acompañada por pruebas que permitan verificar los nuevos requerimientos y los comportamientos críticos modificados. Deberá incluirse evidencia suficiente para demostrar que los cambios de AE2 no



afectan incorrectamente funcionalidades relevantes heredadas de AE1.

Tecnología y lenguaje

El desarrollo podrá realizarse utilizando cualquier lenguaje de programación, framework y conjunto de herramientas aprobado previamente por la cátedra. Por tratarse de una evolución de un sistema existente, se recomienda preservar la compatibilidad con la arquitectura y tecnologías utilizadas en AE1. Cualquier cambio tecnológico significativo deberá ser previamente acordado y fundamentado.

La evaluación no estará centrada en una tecnología particular sino en la **calidad de la solución, las decisiones arquitectónicas, la comunicación entre componentes, la aplicación de los conceptos de la asignatura y las evidencias presentadas.**

Evidencias individuales requeridas

La entrega deberá permitir distinguir con claridad el estado heredado de AE1 de la producción correspondiente a AE2.

Como mínimo deberá presentarse:

- referencia al tag/release/commit utilizado como versión base de AE1
- URL del repositorio
- branch individual de AE2. Descargar del zip del branch realizado por el alumno (para tener evidencia en campus)
- listado o referencia a los requerimientos funcionales seleccionados
- issues o tareas asociadas
- historial de commits individuales
- código implementado
- pruebas
- contratos de APIs y eventos cuando corresponda
- documentación de Redis y RabbitMQ cuando corresponda
- decisiones de arquitectura y persistencia
- instrucciones de ejecución
- README o documentación equivalente actualizada
- Portafolio Digital con evidencias del proceso
- reflexión individual sobre decisiones, dificultades y aprendizajes

Resultado esperado de AE2

Al finalizar la actividad, el estudiante deberá poder demostrar individualmente que es capaz de:

- comprender y evolucionar una aplicación distribuida desarrollada previamente



- transformar requerimientos funcionales en modificaciones concretas de software
- analizar y mejorar límites entre componentes y servicios
- diseñar contratos de comunicación claros
- diferenciar y aplicar comunicación síncrona y asíncrona
- utilizar mecanismos de caché, estado efímero y mensajería cuando corresponda
- resolver problemas de concurrencia, consistencia e idempotencia
- tomar decisiones sobre persistencia e interoperabilidad
- integrar servicios o artefactos externos cuando el escenario lo requiera
- probar y documentar la evolución realizada
- comparar alternativas técnicas y fundamentar las decisiones adoptadas
- mantener trazabilidad entre requerimientos, tareas, código, pruebas y evidencias
- trabajar de manera autónoma sobre una base de software previamente construida en equipo.

La versión resultante de AE2 constituye una **evolución individual del sistema desarrollado en AE1** y podrá ser utilizada posteriormente como evidencia, referencia o insumo para las siguientes actividades de la asignatura.

TIPO DE CALIFICACIÓN:

Formativa, obligatoria y sin nota numérica disciplinar. Registro formal: Entregada / No entregada. Incluye feedback docente y posibilidad de presentar una versión mejorada dentro de los 10 días posteriores a la devolución, cuando corresponda.

Ante dudas razonables respecto de la autoría, participación o apropiación de la producción presentada, la cátedra podrá habilitar una Verificación Oral de Autoría conforme a los lineamientos institucionales. Esta instancia tiene carácter excepcional y no constituye un coloquio adicional ni una nueva evaluación

CONSIGNAS DE ACTIVIDADES DE PROCESO: *Portafolio Digital*



- Mantener evidencias fechadas del proceso: diagramas, decisiones, versiones, capturas, pruebas, logs, métricas y devoluciones.
- Incluir referencias a tareas/issues y commits o pull requests relevantes.
- Registrar alternativas consideradas y el motivo de la decisión adoptada.
- Conservar evidencia de feedback y, en AE1/AE2, de la versión mejorada cuando corresponda

CONSIGNAS DE ACTIVIDADES DE TRABAJO AUTÓNOMO:

- Revisar bibliografía y materiales de la unidad que sustenten decisiones de arquitectura y comunicación.
- Investigar documentación oficial de las herramientas elegidas y registrar las fuentes utilizadas.
- Preparar entorno reproducible de ejecución y pruebas.
- Actualizar documentación técnica a medida que cambia la solución, evitando reconstruirla únicamente al final.

CONSIGNAS DE ACTIVIDADES DE BITÁCORA. *Trabajos grupales*

Por tratarse de una actividad individual, la Bitácora se utiliza como reflexión de proceso: identifique una decisión arquitectónica, compare al menos dos alternativas, explique una dificultad o fallo detectado, describa cómo cambió su solución después del feedback y vincule la reflexión con evidencias del Portafolio.

CONSIGNAS DE ACTIVIDADES DE FORO. *Para asignaturas digitales*

-



Anexo 1: Propuestas de Escenarios

Las siguientes propuestas permiten aplicar la misma consigna y criterios. La cátedra podrá seleccionar una de ellas, modificarla o reemplazarla por otro dominio con complejidad equivalente.

Escenario 1 - Sistema de Compra de Entradas v1 (Parte 1)

Introducción al escenario

Desarrollar un sistema modular para la gestión y venta de entradas a eventos. El sistema debe permitir la visualización de eventos, la compra de entradas mediante una cola controlada, y la administración completa de los eventos y usuarios vía APIs. La solución debe ser responsiva, contenerizada y diseñada para integrarse con mecanismos de autenticación modernos.

Alcance funcional

El sistema estará compuesto por módulos que separan responsabilidades: interfaz pública de eventos, APIs de eventos y compras, gestión administrativa y un servicio de autenticación. Cada módulo se comunicará vía APIs REST bien definidas.

Alcance propuesto v1.0

El alcance propuesto para esta versión 1 del escenario completo

La implementación no debe transformarse en el sistema completo. El equipo debe demostrar una vertical funcional coherente. Los restantes módulos pueden quedar como contratos/stubs aprobados, siempre que exista diseño de los límites de responsabilidad y plan de evolución

ID	Título	Descripción
SI-AE1.1	Catálogo público de eventos	Visualización responsive de eventos, imágenes, fechas, categorías, disponibilidad y navegación.
SI-AE1.2	API de eventos	Listado paginado, detalle, filtros y disponibilidad.
SI-AE1.3	Compra V1	Selección de categoría/cantidad, límite de entradas, cálculo total, medio de pago



		simulado y comprobante con identificador/QR.
S1-AE1.4	API de compras	Reserva básica, compra, límite por usuario/evento y consulta de tickets.
S1-AE1.5	Gestión administrativa	Alta de eventos, fechas, categorías, imágenes y configuración básica.
S1-AE1.6	Autenticación	Registro/inicio de sesión y roles básicos, preparado para OAuth2/OIDC.

Módulo 1 – Aplicación Web Pública de Eventos (V1)

- RF1: Visualización de eventos en cuadrícula, con tarjeta por evento (imagen, título, descripción, precio, fecha).
- RF2: Soporte para múltiples fechas por evento.
- RF3: Inclusión de categorías de eventos y secciones dinámicas (ej.: “Más vendidos”).
- RF4: Estado de disponibilidad de entradas (disponible/agotado).
- RF6: Navegación y acciones básicas: botón de compra simulado y botón de arrepentimiento.
- RF7: Barra de navegación con buscador, enlaces a secciones y gestión de sesión.
- RF8: Footer con información legal, enlaces y redes sociales.

Módulo 2 – API Web de Eventos

- Listar eventos con paginación
- Obtener detalle de un evento con imágenes y categorías.
- Filtros avanzados: fecha, ubicación, categoría y rango de precio.
- Cálculo de disponibilidad en tiempo real

Módulo 3 – Aplicación Web de Compras (V1)

- RF-01: Visualización de datos del evento.
- RF-02: Listado de categorías con precios y stock.
- RF-03: Límite de entradas por categoría.
- RF-04: Cálculo automático del total a pagar.
- RF-05: Selección de método de pago.
- RF-06: Emisión de comprobante de pago con ID de orden y QR.



Módulo 4 – API Web de Compras (V1)

- RF-01: Emisión de comprobantes/tickets.
- RF-02: Compra de entradas de cualquier evento.
- RF-03: Reserva básica de lugares.
- RF-04: Límite máximo de compra por usuario/evento.
- RF-05: Almacenamiento de tickets para consultas posteriores.

Módulo 5 – Gestión de Eventos (Panel Administrativo)

- Desarrollo del formulario completo en /crear con validaciones de campos.
- Integración de Google Maps y Places API para geolocalización y autocompletado.
- Implementación de subida de imágenes (hasta 4 por evento) con galería temática.
- Configuración de TanStack Query para fetch, caché y persistencia.
- Endpoints en Hono Backend conectados a Supabase con Prisma como ORM.
- Integración con Módulo 7 para autenticación/autorización vía HTTPS.

Módulo 6 – API de Gestión de Eventos

- Objetivo: Backend para las operaciones CRUD relacionadas con eventos. (Estado: Conectada a la base de datos, soporta carga de eventos con múltiples fechas y categorías.)

Módulo 7 – API de Autenticación de Usuarios

- Registro e inicio de sesión.
- Roles básicos (usuario / administrador).
- Preparado para soportar OAuth2/OpenID Connect en futuras iteraciones.

PARTE 2

Alcance propuesto v2.0



ID	Título	Descripción
SI-AE2.1	Evolución a microservicios	Reestructurar APIs para reducir acoplamiento y adaptar esquemas de datos.
SI-AE2.2	Cola y concurrencia	Implementar cola de espera, concurrencia controlada y consistencia de reserva.
SI-AE2.3	Cancelaciones e historial	Cancelar reservas, reenviar comprobantes/tickets y consultar historial de compras.
SI-AE2.4	Múltiples divisas	Agregar tratamiento de moneda o estrategia equivalente definida por cátedra.
SI-AE2.5	Administración V2	Edición/cancelación, fechas de visibilidad, visualizaciones, categorización y otras mejoras asignadas.
SI-AE2.6	Autenticación V2	Evolucionar el mecanismo de autenticación y autorización conforme a la arquitectura adoptada.

Módulos completos

Se presenta la descripción integral de los módulos con el propósito de brindar una visión global de la solución y permitir comprender la aplicación como un sistema completo.

No obstante, para esta instancia deberán considerarse específicamente los lineamientos definidos en el apartado “Alcance de la Versión 1.0”, que determinan las funcionalidades comprometidas para la presente entrega.

La descripción general de los módulos tiene carácter orientativo respecto de la evolución esperada del sistema y permite anticipar el alcance de las versiones posteriores. En función del avance del proyecto y de las decisiones pedagógicas y técnicas adoptadas, los estudiantes, en conjunto con el equipo docente, podrán acordar el intercambio, reordenamiento o reasignación de determinadas funcionalidades entre las versiones 1.0 y 2.0, siempre que se mantengan los objetivos de aprendizaje y el alcance general previsto para la actividad.



Módulo 1: Aplicación web pública de eventos

Interfaz para que los usuarios visualicen eventos disponibles.

Cada evento mostrará:

- Al menos una imagen (obligatoria), título y descripción.
- Fechas disponibles (un evento puede tener varias fechas).
- Categorías de entradas (por ejemplo: General, VIP).
- Estado de disponibilidad por categoría (disponible / agotado).
- La interfaz debe ser responsive para uso en dispositivos móviles y de escritorio.
- Desde la vista del evento el usuario podrá iniciar el proceso de compra (redirigiendo a la Aplicación de Compras - Módulo 3).

Módulo 2: API Web de Eventos

- Todas las operaciones de consulta de eventos desde la Aplicación web pública deberán realizarse a través de esta API.
- Endpoints básicos: listar eventos, obtener detalle de evento (incluyendo imágenes y categorías), filtrar por fecha/ubicación/categoría.
- Debe manejar paginación y respuestas con información de disponibilidad en tiempo real.

Módulo 3: Aplicación web de compras

Interfaz para que el usuario realice la compra de entradas.

- Implementación de cola de espera por evento:
- Configurable por evento (por ejemplo: cola de N usuarios simultáneos).
- Si la cola está llena, el sistema informará que no hay lugar hasta que se libere espacio.
- Control de la cantidad máxima por compra (configurable por evento y por categoría).
- Gestión de concurrencia: número de usuarios atendidos concurrentemente (configurable).
- Flujo típico: entrar a la cola → obtener turno → seleccionar categoría y cantidad → confirmar pago.
- Todas las acciones de esta interfaz deben consumir la API de Compras (Módulo 4).

Módulo 4: API Web de Compras

API encargada de todo lo relacionado con el proceso de compra (reserva temporal, validación de stock por categoría, confirmación de pago, emisión de comprobante/entrada).



Debe garantizar consistencia en escenarios concurrentes (evitar sobreventa).

Endpoints recomendados: iniciar/consultar cola, reservar entradas, confirmar compra, cancelar reserva, consultar historial de compras del usuario.

Módulo 5: Gestión de Eventos (Panel administrativo)

Interfaz para usuarios administradores (gestión privada) para:

- Crear/editar/eliminar eventos.
- Subir una o varias imágenes por evento (p. ej. encabezado y galerías).
- Definir título, descripción y metadata del evento.
- Configurar una o varias fechas por evento.
- Crear y configurar categorías de entradas (precio, cantidad disponible, restricciones por comprador).
- Definir fechas de visibilidad de venta: fecha de inicio de ventas y fecha de finalización (fecha posterior a la cual el evento ya no se mostrará en la web pública).
- Configurar parámetros de la cola y concurrencia de compra por evento.

Todas las operaciones administrativas deberán usar la API de Gestión de Eventos (Módulo 6).

Módulo 6: API de Gestión de Eventos

API encargada de las operaciones CRUD sobre eventos y sus entidades relacionadas (fechas, categorías, imágenes, configuración de ventas).

Endpoints para tareas administrativas: publicar/despublicar evento, ajustar stock por categoría, ver logs/estadísticas básicas.

Módulo 7: API de Autenticación de usuarios

Servicio centralizado para todo lo relacionado con usuarios: registro, autenticación, recuperación de contraseña y gestión de perfiles.

Debe soportar protocolos de autenticación estándares: OAuth2 y OpenID Connect.

Todas las aplicaciones (públicas y privadas) usarán este servicio para validar identidades y permisos (roles: usuario, administrador, operador).

Temas técnicos y requisitos no funcionales

- Todas las aplicaciones deberán estar containerizadas (Docker).
- Base de datos: la elección queda a cada grupo de alumnos (indicar la decisión en el informe).
- Control de código: un único repositorio por grupo (proporcionado por la materia).



- Gestión del proyecto: usar GitHub Projects (o similar) para el dashboard de avance (visualizar tareas realizadas y pendientes)
- Cada commits realizado debe estar asociado a cada tarea realizada (Ejemplo #123)
- Seguimiento de errores: los errores deberán registrarse como issues y cerrarse cuando se resuelvan.
- Consistencia y concurrencia
- Seguridad: comunicación por HTTPS, validación de entradas, manejo seguro de tokens y credenciales.

Restricciones Técnicas

- DB: La base de datos debe ser relacional. Se pueden utilizar varias base de datos a decisión del diseño y conveniencia por la arquitectura planteada.
- DB: No se deben realizar ni updates ni delete en la base de datos
- Se debe mantener la versión de las aplicaciones/Módulos (tomar como base el versionado semántico). Ver descripción de versionado



Escenario 2 - Aplicación de Movilidad tipo Uber

Introducción

Construir una plataforma distribuida de movilidad en la que clientes puedan solicitar viajes y conductores habilitados puedan recibir, aceptar o rechazar ofertas. El sistema debe contemplar la cancelación por cliente o conductor, el ciclo completo del viaje y la evolución hacia una arquitectura observable, resiliente y desplegable

Regla tecnológica transversal

El escenario no impone un lenguaje de programación específico. Puede implementarse en Java, TypeScript, C#, Python, Go u otro lenguaje/framework, siempre con aprobación previa de la cátedra. La evaluación se centra en competencias, arquitectura, comunicación, concurrencia, calidad y evidencias, no en una tecnología particular.

Alcance propuesto v1.0

ID	Título	Descripción
S2-AE1.1	Actores y vehículos	Clientes, conductores y vehículos Auto/Moto con autenticación/roles básicos.
S2-AE1.2	Disponibilidad	Conductor se conecta, actualiza posición y queda disponible para solicitudes.
S2-AE1.3	Solicitud y despacho básico	Cliente indica origen/destino/tipo; se busca conductor y se envía oferta.
S2-AE1.4	Aceptar, rechazar y cancelar	Conductor acepta/rechaza; cliente puede cancelar; sólo un conductor queda asignado.
S2-AE1.5	Ciclo de viaje	Estados mínimos solicitado/asignado/en curso/completado/cancelado.
S2-AE1.6	Tarifa y comprobante simple	Estimación básica y comprobante de demostración sin pasarela real.

Condición del alcance EA1



La implementación no debe transformarse en el sistema completo. El equipo debe demostrar una vertical funcional coherente. Los restantes módulos pueden quedar como contratos/stubs aprobados, siempre que exista diseño de los límites de responsabilidad y plan de evolución

Alcance propuesto v2.0

ID	Título	Descripción
S2-AE2.1	Redis para ubicación/despacho	Ubicación de conductores y ofertas temporales con TTL.
S2-AE2.2	RabbitMQ	Publicar/consumir eventos de viaje, pago y notificación.
S2-AE2.3	Concurrencia e idempotencia	Resolver carreras de aceptación, cancelaciones simultáneas y reintentos.
S2-AE2.4	QR de inicio	QR temporal de un solo uso para validar comienzo de viaje.
S2-AE2.5	PDF de comprobante	Generar y almacenar comprobante PDF posterior al viaje/pago.
S2-AE2.6	Integraciones externas	Usar o simular proveedor de mapas y pasarela de pagos con adaptación y manejo de fallos.

Actores y conceptos principales

- **Cliente:** persona que solicita un viaje, selecciona Auto/Moto, puede cancelar y calificar.
- **Conductor:** persona habilitada que publica disponibilidad, recibe ofertas y puede aceptar, rechazar o cancelar.
- **Vehículo:** Auto o Moto habilitado para prestar un tipo de servicio.
- **Viaje:** entidad de negocio con ciclo de vida controlado y trazable.
- **Operador/Administrador:** rol de soporte para habilitaciones, bloqueos y consulta operativa.
- **Servicios externos:** mapas/geocoding, pasarela de pagos y canales de notificación, reales o sandbox según la etapa.



Módulos completos

Se presenta la descripción integral de los módulos con el propósito de brindar una visión global de la solución y permitir comprender la aplicación como un sistema completo.

No obstante, para esta instancia deberán considerarse específicamente los lineamientos definidos en el apartado “Alcance de la Versión 1.0”, que determinan las funcionalidades comprometidas para la presente entrega.

La descripción general de los módulos tiene carácter orientativo respecto de la evolución esperada del sistema y permite anticipar el alcance de las versiones posteriores. En función del avance del proyecto y de las decisiones pedagógicas y técnicas adoptadas, los estudiantes, en conjunto con el equipo docente, podrán acordar el intercambio, reordenamiento o reasignación de determinadas funcionalidades entre las versiones 1.0 y 2.0, siempre que se mantengan los objetivos de aprendizaje y el alcance general previsto para la actividad.

Estrategia evolutiva de datos

AE1 puede utilizar una misma instancia física de base de datos para reducir complejidad, siempre separando esquemas y documentando propiedad. Por ejemplo, M2/M3 pueden compartir una instancia “Profiles” y M5/M6 una “Mobility”. En AE2 debe justificarse la separación lógica y queda prohibido consultar directamente tablas de otro servicio. Para AE4 se recomienda database-per-service o una excepción explícitamente fundamentada. Esta progresión permite discutir autonomía, transacciones distribuidas y acoplamiento sin convertir la primera actividad en un ejercicio de infraestructura.

ID	Servicio	Responsabilidad	Persistencia recomendada	Infra/externos
M1	Identidad y Acceso	Registro, autenticación, recuperación de acceso, roles y permisos para clientes, conductores y operadores. Emite/valida tokens y desacopla identidad del resto del dominio.	IdentityDB separada	IdP/OAuth2-OIDC; caché opcional de metadatos



M2	Clientes	Gestiona perfil del cliente, direcciones frecuentes, preferencias, historial resumido y calificaciones emitidas. No accede directamente a las tablas de viajes o pagos.	CustomerDB separada (AE1 puede compartir instancia física, esquema aislado)	Cache Redis opcional de perfil
M3	Conductores y Vehículos	Administra datos del conductor, habilitación, vehículo, tipo Auto/Moto, documentación, estado operativo y calificaciones.	DriverDB separada	Servicio externo opcional de validación documental
M4	Ubicación y Disponibilidad	Recibe posición de conductores, mantiene disponibilidad de corto plazo, consulta conductores cercanos y estima distancia/ETA.	LocationDB opcional para histórico; estado caliente fuera de SQL	Redis obligatorio desde AE2; proveedor de mapas/geocoding
M5	Solicitud y Despacho	Recibe solicitudes de viaje, busca candidatos, envía ofertas con vencimiento y asegura que un viaje sea asignado a un único conductor.	DispatchDB; en AE1 puede compartir MobilityDB con M6, luego justificar separación	Redis para ofertas/TTL; RabbitMQ desde AE2
M6	Viajes y Ciclo de Vida	Administra la máquina de estados del viaje: solicitado, asignado, conductor en camino, arribado, en curso, completado o cancelado. Registra	TripDB; en AE1 puede compartir MobilityDB con M5	RabbitMQ; validación de QR con M8



		transiciones y reglas de cancelación.		
M7	Tarifas, Pagos y Liquidaciones	Calcula estimación, registra método de pago, autoriza/captura cobros, aplica cargos de cancelación, reintegros y liquidaciones.	PaymentsDB separada, con historial auditable	Pasarela de pago sandbox/externa; RabbitMQ
M8	Notificaciones, Documentos y Soporte	Envía notificaciones, genera QR de verificación, genera comprobante PDF, permite reenvío y registra tickets de soporte asociados al viaje.	CommunicationsDB separada + almacenamiento de objetos para PDF	RabbitMQ; email/SMS/push; generador QR/PDF

Requerimientos Funcionales

M1 - Identidad y Acceso

ID	Título	Descripción
RF-1.1	Registro de identidad	Permitir registrar clientes y conductores con los datos mínimos definidos por la cátedra.
RF-1.2	Autenticación	Permitir iniciar sesión y obtener una credencial/ticket de acceso para consumir los servicios protegidos.
RF-1.3	Roles y permisos	Diferenciar, como mínimo, Cliente, Conductor y Operador/Administrador.



RF-1.4	Recuperación y revocación	Permitir recuperación de acceso y revocación de credenciales cuando corresponda.
RF-1.5	Integración estándar	La evolución del escenario debe soportar OAuth2/OpenID Connect o un mecanismo equivalente aprobado por la cátedra.

M2 - Clientes

ID	Título	Descripción
RF-2.1	Perfil de cliente	Crear y consultar los datos del cliente y sus preferencias.
RF-2.2	Direcciones frecuentes	Registrar origen/destino favoritos o recientes.
RF-2.3	Historial de viajes	Consultar los viajes del cliente mediante APIs, sin acceso directo a la DB del servicio de viajes.
RF-2.4	Calificación del conductor	Permitir registrar una valoración posterior a un viaje completado.
RF-2.5	Estado de cuenta	Consultar información básica del estado del perfil y bloqueos administrativos.

M3 - Conductores y Vehículos

ID	Título	Descripción
RF-3.1	Perfil de conductor	Crear y consultar datos del conductor y su estado de habilitación.
RF-3.2	Vehículos Auto/Moto	Registrar uno o más vehículos e indicar el tipo de servicio habilitado: Auto y/o Moto.
RF-3.3	Disponibilidad	Permitir al conductor declararse conectado/desconectado y disponible/no disponible.



RF-3.4	Documentación		Registrar metadatos de licencia, seguro y documentación del vehículo.
RF-3.5	Calificación cliente	del	Permitir al conductor valorar al cliente al finalizar el viaje.

M4 - Ubicación y Disponibilidad

ID	Título		Descripción
RF-4.1	Actualización de ubicación		Recibir coordenadas del conductor con marca temporal mientras esté conectado.
RF-4.2	Conductores cercanos		Consultar conductores disponibles próximos a un origen y compatibles con Auto/Moto.
RF-4.3	Vencimiento de ubicación		No considerar ubicaciones antiguas; utilizar TTL para evitar conductores fantasma.
RF-4.4	Geocodificación		Resolver direcciones y coordenadas mediante proveedor externo o servicio simulado.
RF-4.5	ETA y distancia		Obtener una estimación de distancia/tiempo para apoyar despacho y tarifa.

M5 - Solicitud y Despacho

ID	Título		Descripción
RF-5.1	Solicitud de viaje		El cliente solicita un viaje indicando origen, destino y tipo Auto/Moto.
RF-5.2	Búsqueda candidatos	de	Seleccionar conductores disponibles conforme a proximidad y tipo de vehículo.
RF-5.3	Oferta vencimiento	con	Enviar una oferta a uno o más conductores con tiempo máximo de respuesta.



RF-5.4	Aceptar o rechazar	El conductor puede aceptar o rechazar una oferta mientras esté vigente.
RF-5.5	Asignación única	Ante respuestas concurrentes, sólo un conductor puede quedar asignado al viaje.
RF-5.6	Cancelación previa	El cliente puede cancelar la solicitud antes de la asignación o según las reglas definidas.
RF-5.7	Sin disponibilidad	Informar cuando no existan conductores adecuados o expire el proceso de búsqueda.
RF-5.8	Trazabilidad del despacho	Registrar eventos relevantes de oferta, rechazo, timeout, aceptación y cancelación.

M6 - Viajes y Ciclo de Vida

ID	Título	Descripción
RF-6.1	Estados del viaje	Administrar estados válidos y transiciones controladas del viaje.
RF-6.2	Arribo del conductor	Permitir al conductor indicar que llegó al punto de retiro.
RF-6.3	Inicio validado	Iniciar el viaje sólo con una condición de verificación definida; desde AE2 se propone QR de un solo uso.
RF-6.4	Finalización	Finalizar registrando tiempo, distancia y datos necesarios para tarifa/pago.
RF-6.5	Cancelación cliente por	Permitir cancelar por el cliente con motivo, estado y eventual cargo.
RF-6.6	Cancelación conductor por	Permitir cancelar por el conductor con motivo y retorno del cliente al proceso de despacho cuando corresponda.
RF-6.7	Historial de transiciones	Mantener auditoría de las transiciones del viaje sin editar la historia ya registrada.



M7 - Tarifas, Pagos y Liquidaciones

ID	Título	Descripción
RF-7.1	Estimación de tarifa	Calcular una estimación basada en tipo de vehículo, distancia, tiempo u otros parámetros configurables.
RF-7.2	Método de pago	Registrar o seleccionar un medio de pago permitido por el escenario.
RF-7.3	Autorización/captura	Simular o integrar autorización y captura de pago al completar el viaje.
RF-7.4	Cargo de cancelación	Calcular un cargo de cancelación cuando las reglas del viaje lo indiquen.
RF-7.5	Reintegro	Registrar un reintegro total o parcial ante una cancelación válida posterior al cobro.
RF-7.6	Idempotencia de pago	La repetición de una misma orden de cobro no debe generar cobros duplicados.
RF-7.7	Historial financiero	Mantener trazabilidad de operaciones financieras y su estado.

M8 - Notificaciones, Documentos y Soporte

ID	Título	Descripción
RF-8.1	Notificaciones de viaje	Notificar solicitud, asignación, arribo, inicio, cancelación y finalización por uno o más canales.
RF-8.2	QR de verificación	Generar un QR temporal y de un solo uso asociado al viaje para validar el inicio, sin exponer información sensible.
RF-8.3	Comprobante PDF	Generar un PDF posterior a la finalización/pago con datos de viaje, importe, identificadores y fecha.



RF-8.4	Reenvío de comprobante	Permitir volver a solicitar el enlace o envío del comprobante.
RF-8.5	Soporte asociado a viaje	Crear un ticket de soporte asociado a un viaje y registrar su estado.
RF-8.6	Consumo asíncrono	Procesar eventos desde RabbitMQ para evitar que notificaciones/documentos bloqueen el flujo principal.

Requerimientos no funcionales y técnicos

Requerimientos no funcionales

ID	Título	Descripción	Nivel mínimo
RNF-01	Libertad tecnológica con aprobación	El desarrollo puede realizarse en cualquier lenguaje y framework elegido por el alumno o grupo, con aprobación previa de la cátedra. La elección debe justificarse por adecuación al problema, herramientas, interoperabilidad y capacidad de despliegue.	AE1
RNF-02	Arquitectura modular por servicios	Los ocho módulos deben exponerse como servicios/microservicios con responsabilidad explícita. No se permite resolver todo el dominio como una única aplicación monolítica sin justificar una excepción.	AE1
RNF-03	Contenerización OCI	Cada aplicación/servicio y dependencia local debe poder ejecutarse en contenedores mediante Docker, Podman o equivalente aprobado.	AE1
RNF-04	Propiedad de datos	Cada servicio debe documentar si posee DB propia, comparte una instancia física con esquema aislado o comparte una DB por decisión	AE1→AE2



		transitoria. Desde AE2 no debe consultar directamente tablas pertenecientes a otro servicio.	
RNF-05	APIs y contratos	Las comunicaciones síncronas deben usar contratos bien definidos; se recomienda REST/HTTP y documentación OpenAPI. Los errores deben tener formato consistente.	AE1
RNF-06	Cache y estado efímero con Redis	Redis será obligatorio desde AE2 para ubicación/disponibilidad, ofertas con TTL y/o caché de información de alta lectura. Debe evidenciarse uso correcto, expiración e invalidación.	AE2
RNF-07	Mensajería con RabbitMQ	RabbitMQ será obligatorio desde AE2 para al menos dos flujos asíncronos, por ejemplo TripAccepted, TripCompleted, PaymentConfirmed o NotificationRequested.	AE2
RNF-08	Idempotencia	Consumidores de mensajes y operaciones críticas como pago, cancelación o finalización deben tolerar reintentos sin duplicar efectos.	AE2
RNF-09	Consistencia y concurrencia	El sistema debe impedir doble asignación de conductor, doble finalización, doble cobro y transiciones inválidas aunque existan solicitudes concurrentes.	AE1→AE4
RNF-10	Seguridad de transporte y acceso	Usar HTTPS en ambientes publicados; validar entradas; proteger endpoints; gestionar tokens/credenciales de forma segura; no almacenar secretos en el código.	AE2→AE4
RNF-11	Configuración externa	Variables de entorno/secret manager para configuración. No codificar claves, cadenas de conexión o secretos en repositorio.	AE1



RNF-12	Usabilidad y adaptabilidad	Las interfaces propuestas deben ser responsivas y mostrar estados claros para espera, aceptación, cancelación, error y modo degradado.	AE1
RNF-13	Logs y correlación	Logs estructurados con identificador de correlación/tripId para reconstruir un flujo distribuido.	AE2
RNF-14	Resiliencia	Aplicar timeouts, reintentos controlados y Circuit Breaker en dependencias críticas. El fallo de un servicio no debe colgar la interfaz ni producir errores sin tratamiento.	AE4
RNF-15	Alta disponibilidad	Ejecutar al menos dos réplicas de uno o más servicios críticos y demostrar continuidad ante caída de una réplica mediante proxy/balanceador y healthchecks.	AE4
RNF-16	Health checks	Exponer endpoints de salud/liveness/readiness y distinguir dependencia disponible/degradada cuando corresponda.	AE2→AE4
RNF-17	Testing	Incluir pruebas de unidad e integración desde AE1; en AE4 incorporar pruebas de sistema, carga/concurrencia, seguridad y casos de resiliencia.	AE1→AE4
RNF-18	Performance	Definir metas medibles para endpoints/flujo crítico y ejecutar pruebas de carga sobre solicitud/aceptación de viajes.	AE4
RNF-19	Observabilidad	En AE4 incorporar métricas, logs y trazas. Se propone Prometheus/Grafana y trazabilidad de al menos dos recorridos críticos de punta a punta.	AE4



RNF-20	CI/CD	Automatizar build, tests, imágenes y publicación en registry; al menos una entrega debe desplegarse sin intervención manual después de la aprobación del pipeline.	AE4
RNF-21	Publicación	La solución integradora debe estar accesible en Internet o en infraestructura de demostración aprobada por la cátedra.	AE4
RNF-22	Versionado gestión y	Versionado semántico, issues/tareas, commits trazables, releases y README con instrucciones de ejecución.	AE1→AE4
RNF-23	Privacidad auditoría y	Evitar exposición de datos personales en logs, QR o URLs; conservar trazabilidad de operaciones críticas y justificar política de retención.	AE4

Progresión por Actividad de Evaluación

Actividad	Nivel	Alcance técnico-funcional	Foco del Programa 2026
AE1	Base funcional inmersión	Definir los 8 límites de servicio; implementar una vertical funcional mínima que permita registrar actores, publicar disponibilidad, solicitar un viaje Auto/Moto, aceptar/rechazar y cancelar. Comunicación principalmente REST. Contenedores, repositorio, Portafolio y Bitácora. Los servicios no terminados pueden	Ejes 1 y 2: concurrencia/distribución inicial + desarrollo web.



		presentarse con contrato + stub aprobado.	
AE2	Profundización arquitectónica	Trabajo individual sobre una base congelada/provista. Incorporar Redis y RabbitMQ, separar responsabilidades y datos, manejar concurrencia/idempotencia, evolución del flujo, QR/PDF y un servicio externo o sandbox. Documentar decisiones y comparar alternativas.	Eje 3: arquitectura, comunicación, REST, asincronía, SOA e interoperabilidad.
AE4	Integración y producción	Trabajo en pareja. Integrar los 8 servicios y demostrar el recorrido completo con seguridad, testing, resiliencia, HA, CI/CD, observabilidad, OpenAPI y publicación. Incluir análisis de un cuello de botella y propuesta de mejora. Coloquio obligatorio con defensa individual.	Integra Ejes 1–4 y TIFAS.

Alcance mínimo sugerido para AE1

- Definir los ocho servicios y sus contratos, aun cuando algunos se presenten como stub en la primera versión.
- Implementar una vertical funcional con M1–M6 y simulación simple de M7/M8.
- Permitir: registrar actores, publicar disponibilidad, solicitar Auto/Moto, ofrecer viaje, aceptar/rechazar, cancelar y completar un viaje básico.
- Demostrar una situación de concurrencia (por ejemplo, dos conductores intentando aceptar la misma solicitud) y cómo se resuelve.
- Contenerizar y documentar ejecución; usar repositorio, tablero/issues, Portafolio y Bitácora individual.
- No es obligatorio RabbitMQ en AE1; Redis puede introducirse de forma opcional o guiada para ubicación.

10. Alcance mínimo sugerido para AE2



- Incorporar Redis obligatoriamente para ubicación/ofertas/caché según diseño.
- Incorporar RabbitMQ para al menos dos flujos asíncronos y documentar productor, consumidor, evento y estrategia de reintentos.
- Separar o justificar la persistencia por servicio; no realizar lecturas SQL cruzadas entre servicios.
- Resolver idempotencia y al menos un caso de carrera realista: aceptación simultánea, cancelación vs. aceptación o doble cobro.
- Generar QR temporal de validación y PDF de comprobante; ambos deben evitar exposición de datos sensibles.
- Integrar o simular mapas y pasarela de pagos; aplicar timeouts y manejo explícito de error básico.
- Documentar OpenAPI, decisiones y pruebas.

11. Alcance mínimo sugerido para AE4 / TIFAS

- Integrar los ocho servicios y demostrar dos recorridos críticos de punta a punta.
- Contenerización completa, healthchecks y red interna de servicios.
- Redis y RabbitMQ operativos, con evidencia de expiración, mensajes, reintentos e idempotencia.
- Seguridad, controles de autorización y gestión de secretos.
- Alta disponibilidad y prueba de caída de al menos un servicio crítico.
- Resiliencia mediante timeouts, reintentos y Circuit Breaker o patrón equivalente.
- CI/CD, registry, versionado y publicación en entorno aprobado.
- Observabilidad con logs estructurados, métricas y trazas; se propone Prometheus/Grafana u opción equivalente aprobada.
- Pruebas de unidad, integración, sistema, carga/concurrencia y seguridad apropiadas.
- Analizar un cuello de botella cuantificable (por ejemplo, tiempo de asignación o tasa de abandono) y proponer una mejora/MVP.

Evidencias esperadas

- Repositorio Git con historial y releases.
- README reproducible y diagrama de componentes/servicios.
- OpenAPI y catálogo de eventos asíncronos.
- Diagrama de datos/propiedad por servicio y ADRs o decisiones de arquitectura.
- Pruebas y logs/capturas/métricas.
- Portafolio de evolución y Bitácora individual cuando corresponda.
- Demostración de QR y PDF en las etapas donde sean requeridos.

Criterios de aceptación del escenario



- Los requisitos seleccionados para cada AE son trazables a una evidencia concreta.
- Las decisiones de microservicios y datos son justificadas, no sólo dibujadas.
- Se puede ejecutar la solución con instrucciones reproducibles.
- Los flujos concurrentes no generan doble asignación, doble cobro ni estados imposibles.
- Los fallos de servicios externos se tratan explícitamente.
- El escenario puede adaptarse por la cátedra sin cambiar las competencias evaluadas.